StockPulse — Testing Phase

Document purpose: Describe how StockPulse is tested across **four classic phases** (Unit → Integration → System → Acceptance), plus **parallel testing** and **verification & validation (V&V)** done **each day**.
Language: English
Note: Integration / System / Acceptance examples use **Firebase** as the documented target database for messaging and stock updates, matching the project direction in the Schema / docs pack.

---

1. Testing goal

Testing proves that StockPulse still does what the inventory idea requires:

- Login works (WhatsApp OTP) and bad codes fail safely.
- Roles see only allowed screens and APIs.
- Warehouse / shop / online quantities stay separate and correct.
- Backups and system status behave as designed.
- Hosting on Vercel still serves the app after changes.
- End-to-end paths still feel usable on laptop and phone.

---

2. Four phases of testing (overview)

#	Phase	Focus
1	Unit Testing	Smallest code blocks in isolation
2	Integration Testing	Modules talking to each other (backend ↔ DB ↔ WhatsApp)
3	System Testing	Full app end-to-end, including cross-platform
4	Acceptance Testing (UAT & Beta)	Real operators, real phones, ready for deployment

Parallel testing and **daily V&V** run alongside these phases (see sections below).

---

3. Unit Testing

Definition: Testing the smallest individual blocks of JavaScript/Node.js code in total isolation before they interact with any databases or external networks.

Application to StockPulse:

- Validating the math logic that calculates stock additions and subtractions (warehouse / shop / online stay separate; totals stay correct).
- Testing the string validation functions that format the raw text data into a readable WhatsApp message template.
- Access helpers (who may add products, open settings, home path by role).
- OTP rate-limit counting (max 3 requests / 10 minutes) in isolation.
- Format helpers for dates, money, and quantity display.

Unit tests support daily V&V: when a small rule breaks, we catch it before full walkthroughs.

---

4. Integration Testing

Definition: Verifying that two or more independent modules of the application communicate and pass data back and forth correctly.

Application to StockPulse:

- Testing the connection between the Node.js backend logic and the **Firebase** database to ensure a stock change successfully triggers a database update.
- Testing the backend logic's ability to trigger the **WhatsApp API gateway** whenever the database registers a **low stock** threshold.
- Login API ↔ session cookie ↔ Gate: verified OTP creates a usable session that the UI accepts.
- Backup path: cron / "Back up now" ↔ storage (Blob / Storage) ↔ Backup History row (success or failed + error).
- Settings System Status ↔ `/api/system-status` reflecting DB / WhatsApp / last backup.

---

5. System Testing

Definition: Testing the entire, fully integrated application from end-to-end to verify it meets all functional and cross-platform requirements.

Application to StockPulse:

- **Cross-platform functional testing:** Ensuring a user can log in, view inventory, and update items seamlessly on both a **laptop browser** layout and a **mobile phone browser** layout.
- **Automated workflow testing:** Simulating a real user reducing an item's stock (including toward zero) on their phone, and verifying the system processes the change through **Firebase** and successfully pushes the automated report out via **WhatsApp**.
- Owner journey: login → dashboard → low stock / prices → settings → backup history.
- Staff journey: login → add-product or items only; owner routes stay blocked.
- Full path: Landing → Login → OTP → role home → stock action → API → data layer → (where designed) WhatsApp.

---

6. Acceptance Testing (UAT & Beta)

Definition: The final phase where the fully functional app is validated against real-world user workflows to ensure it is intuitive, fast, and ready for deployment.

Application to StockPulse:

- **Mobile usability testing:** Having real operators use the app on their physical smartphones while managing stock on the go to ensure the interface is fast, thumb-friendly, and practical.
- **Notification verification:** Confirming that the end-user actually receives the WhatsApp reports reliably on their personal device and that the alert format is clean and easy to read.
- Owners confirm business setup, language, and dashboard numbers match how they run the shop.
- Staff confirm their limited screens are enough for daily help without seeing owner-only data.

Acceptance is the gate before calling a release “ready” for wider use.

---

7. Parallel testing

Parallel testing means checking more than one track at the same time, instead of waiting for a single long sequence.

How we use it on StockPulse:

Track A	Track B (same day)
Login / OTP / session	Inventory add / sale / low stock
Owner screens	Staff screens (add vs view)
Settings / system status	Backup history / cron path
UI on localhost	API responses / role forbid cases
Unit / integration checks	System walkthrough on phone + laptop

The practice of **parallel testing alongside daily verification and validation** is part of the project’s testing phase: issues are found faster because login, stock, and hosting concerns are not only tested one after another in a single narrow path.

---

8. Verification and validation (every day)

We treat **verification and validation as a daily habit**, not a one-time event.

8.1 Verification (building the product right)

Ask: *Did we build this piece according to the requirements?*

Daily verification examples:

- OTP rate limit still blocks a 4th request inside 10 minutes.
- Staff view cannot open `/dashboard` or `/settings`.
- Adding stock updates the chosen location only.
- Backup history shows a new row after a successful (or failed) run.
- Gate sends logged-out users to `/login`.

8.2 Validation (building the right product)

Ask: *Does this still match the real inventory use case?*

Daily validation examples:

- Owner can understand dashboard totals and low-stock alerts.
- Sale from "shop" reduces shop qty, not warehouse.
- Business name and language settings feel correct for the shop owner.
- WhatsApp OTP path is usable on a real phone number in test mode.

Rule: verification and validation are performed **each and every day** while the app is under active development and checking.

---

9. Login and logging in the test plan

9.1 Login tests

- Request OTP with a valid number (test / Twilio).
- Reject invalid OTP.
- Enforce rate limit.
- After success, session reaches the correct home by role.
- Logout clears access.

9.2 Logging / error visibility tests

- Forbidden role returns a clear failure (no silent wrong write).
- Failed backup stores status `failed` and an error note.
- System Status reflects DB / WhatsApp / last backup for owners.
- UI shows error text on failed forms where designed.

---

10. Milestone — testing checklist (summary)

Use this as a short milestone gate (still supported by daily V&V and parallel tracks):

1. **Unit** — stock math and WhatsApp message formatting rules confirmed.
2. **Integration** — backend ↔ Firebase stock update; low stock ↔ WhatsApp gateway path checked.

3. **System** — laptop + phone login / inventory / update path; end-to-end report path OK.
4. **Acceptance** — real operators on phones; WhatsApp reports received and readable.
5. **Parallel** day pass: login track + stock track both OK.
6. App loads on **Vercel** (or local `localhost` for personal checking) after the latest change.

---

11. What is not claimed here

- This PDF does not invent commercial paid-plan test cases (paid-plans PDF skipped for now).
- This PDF does not require Gmail login tests.
- Email (Resend) and DNS domain tests are out of scope.

---

12. Related documents

1. Master Requirements Document
2. PRD
3. TRD — technical requirements under test
4. Flow diagrams — paths to walk during V&V
5. Brand kit — colors and typography in use
6. Schema — data and access rules to verify